

Why does your AI hate my handwriting?: Explaining handwriting and suggesting improvements

National University of
Singapore

National University of
Singapore

National University of
Singapore

National University of
Singapore

National University of
Singapore

ABSTRACT

What makes good handwriting? If the aesthetic judgment of handwriting follows implicit rules, can we recover those rules by observing good and bad examples? To answer this question, we applied explainability techniques to the classification of good and bad handwriting. We labelled single-character and word-level datasets using known rules for handwriting, trained classifiers on the data, then explained individual sample predictions to recover the original rules. To provide explanations, we used a modified version of the LIME Image Explainer and visualised the segments of handwriting to alter or keep the same. With this, we generated interpretable suggestion images showing why handwriting is good or bad, and how to improve handwriting.

Keywords

Handwriting; Explainability; Feature Attribution

1. INTRODUCTION

Analysis of handwriting is a well studied and classic problem in computer vision. Early image recognition tools were used to classify handwritten numbers [12], and more recent studies have shown that autism and medical issues can be detected through an individual's handwriting [8].

While two sets of handwritten material may convey the same objective meaning, some handwriting will be visually pleasing and satisfying to look at, while others might appear ugly to some, or be close to illegible. In this study, we work towards quantifying this subjective human perception of “beauty” and “ugliness” in handwriting, using machine learning techniques. Essentially, “*What makes some handwriting visually pleasing, while others appear ugly?*”.

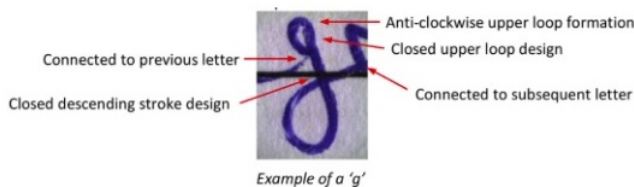


Figure 1: Construction features for a character[2]

While previous studies have scored the legibility and conformity of handwriting, we created a system to interpret the features that lead to the handwriting being classified as good or bad. By learning *why* handwriting is classified as bad, we can understand about the aspects to change for making it good. The results of this research can aid in schools when teaching handwriting, and aid in typographic design. Additionally, individuals can have increased awareness of how their handwriting could be deficient, and assist in self-improvement.

So the research questions in this paper are:

RQ1: Can we build an explainer to infer how a user should change their handwriting to progress from “ugly” to “neat”?

RQ2: What are the key differences between these two classes?

To answer **RQ1**, an image classification algorithm is trained on a data-set of “ugly” and “neat” handwriting. Once text is presented and classified, we used LIME [14], an explainer for machine learning classifiers, to extract the segments in order to quantify why some text is classified as “ugly” while others are classified as “neat”. Qualitative analysis was then used to infer the differences between these two classes.

To answer **RQ2**, we generated a data set which covers the extreme examples of “ugly” and “neat” handwriting which is detailed in 3.1.2. This is to testify if the classifier could capture the features that humans tend to evaluate.

2. RELATED WORK

There has been some existing analysis work on aesthetic features of handwriting. Agius et al. used logistic regression, and classification and regression tree (CRT) [3] to model the relationship between the nationality of the writer and handwriting features of 7 characters (e.g. the letter “g” in **Figure 1**). However, the framework presented in this paper is based on numerical data, which is manually coded. Computer vision is not taken into account to automatically extract the construction features for the testing data. It also did not consider explicit features, such as slant and relative character sizes, which could contribute to the classification results. Handwriting beautification of digital ink has been done on stroke level leveraging on recurrent neural network (RNN) cells[4]. The stroke samples relies on a large database of writings from a user[25].

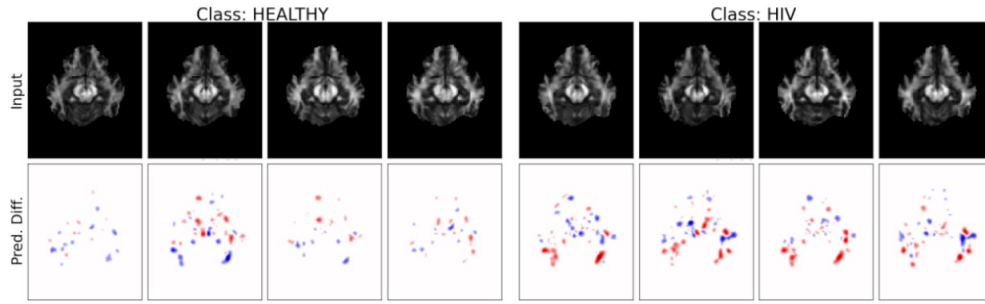


Figure 2: HIV diagnoses from MRI scans where the left-hand images are healthy and the right-hand images have HIV. Areas highlighted in red show evidence for the classification, and blue show evidence against the classification [12]

Additionally there have been a number of studies to score the handwriting of individuals. Falk et al.[7] conducted a study where students used a digital pen to write a series of words. The centre of mass of each letter was then used to compute the size, space and alignment between words, and students were scored according to the Minnesota Handwriting Assessment[14]. Bouillon et al. used a neural network to compare teacher’s ideal letters to those of the student and rate similarity percentage[6]. While these studies feedback to the user a score for their handwriting, they do not say *why* it was scored as good or bad.

Yet explainability for these classification methods has been a popular research direction recently. Shrikumar et al. developed the DeepLIFT (Deep Learning Important Features) method [19], which explains a neural network’s output classification for a specific input, by “back-propagating the contributions of all neurons in the network to every feature of the input”. Zintgraf et al. developed the prediction difference analysis method for deep neural networks [24], which provides visual explanations to how classification is reached for an input image. As seen in **Figure 2**, areas highlighted in red show positive evidence for the classification, while areas in blue show evidence against the classification. A similar visual explainability technique was developed by Sundararajan et al. [21]. Such techniques could be used to explain why handwritten text is classified the way it is.

3. TECHNICAL APPROACH

3.1 Data

To build and verify our classification model, we used an existing well studied data set within computer vision, as well as word-based data sets which closely match our ideal use case. Both data sets followed the “chalkboard” convention of white writing on black background. We labelled these data sets using set criteria, explained further in each case below.

3.1.1 MNIST

We used the MNIST handwritten digits data set, as it is a classic data set within the image recognition community, and consists of thousands of samples of handwritten numbers. Due to its high adoption, there are pre-existing examples of frequently incorrectly classified MNIST samples within literature for reference [20, 5].

As we could not find an extensively labelled data-set of

“good” and “bad” numbers, we labelled the sixes within MNIST using the criteria for “bad” samples, shown in **Figure 3**.

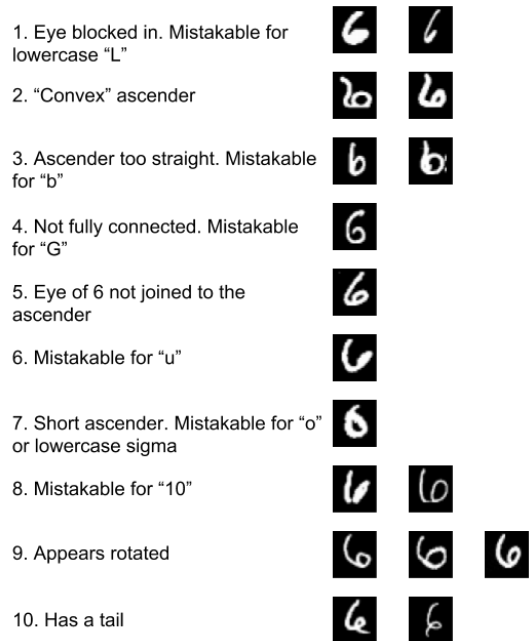


Figure 3: Rules and examples of MNIST samples labelled “bad”

We labelled 4,100 of the 6,875 samples of the number 6 within the MNIST data set. For completeness, examples of sixes labelled “good” are shown in **Figure 4**.



Figure 4: Examples of MNIST samples labelled “good”

3.1.2 Handwritten Word Dataset

We trained a model using multiple samples of a single handwritten word. To label words as “good” or “bad”, we adhered to the Minnesota Handwriting Assessment (MHA)[14],

as it is used widely within the education sector[17, 10, 15]. MHA outlines the following guidelines¹ when assessing handwriting:

Legibility: Is the word readable?

Form: Do the letters look as they are supposed to? Are lines connected and curved/straight in the correct way?

Alignment: Are the characters aligned level?

Size: Are the characters all the same size?

Spacing: Are the characters spaced evenly?

A 438-sample data set of the handwritten word “quickly”[11] was used initially to simulate the problem. The data set was labeled by 5 participants independently and the final label was a mode of the votes. Because of the imbalanced data set (73.7% of samples were labeled “good”) and the limited number of samples, we further generated a data set of the word “studio” according to the following criteria in **Figure 5**.

- | | |
|--|---|
| 1. A good handwriting which has medium spacing and the letters are straight |  |
| 2. A good handwriting which has medium spacing and slants to the left. |  |
| 3. A good handwriting which has medium spacing and slants to the right |  |
| 4. A good handwriting which has narrow spacing and the letters are straight. |  |
| 5. A good handwriting which has wide spacing and the letters are straight. |  |
| 6. A bad handwriting which has illegible letter(s) |  |
| 7. A bad handwriting which has letter(s) with uneven sizes. |  |
| 8. A bad handwriting which has uneven spacing between the letters. |  |
| 9. A bad handwriting which is too squeezed. |  |
| 10. A bad handwriting which has letter(s) not aligned vertically. |  |

Figure 5: Rules and examples of handwritten word “studio” samples

The word “studio” was chosen for the following reasons:

- It has no letters that descend below the line, making the images slightly cleaner.
- It has some confusable letters that can be mistaken for others, e.g. ‘u’ for ‘a’ or ‘n’, and ‘d’ for ‘l’.
- The letters generally had only one or a few correct forms, unlike ‘y’ and ‘q’ in quickly, which were sometimes written with a flourish.

981 “studio” samples were produced manually by hand, with 435 good samples and 483 bad samples. Samples were consequently denoised by Gibbs sampling, as well as adjusted to black and white.

¹We excluded “rate” as we do not know the rate at which a word was written

3.2 Training Our Model

For each data set, we trained a generic convolutional neural network due to their accuracy in classification, well-supported libraries and use for explainability within computer vision. In each case, the training and validation accuracies of the classifiers were all within 89-91

3.3 Explainer: LIME

Local Interpretable Model-Agnostic Explanations (LIME)[16] was used to explain our trained models. In each case, we aim to capture the top regions, or superpixels, that contribute positively or negatively to the two classes via the explainer. LIME fits a local linear decision boundary by perturbing the interpretable components around the sample’s neighborhood. For image explanation, the interpretable components are the segmented superpixels, which are turned on and off during perturbation. The underlining image processing library for LIME is scikit-image which contains 4 choices of segmentation and superpixel algorithms: Felzenszwalb’s method, Quickshift, SLIC, and Compact watershed [18]. The size of the superpixels and the type of the segmenter play an important role in the explanation. The details are in 3.3.2.

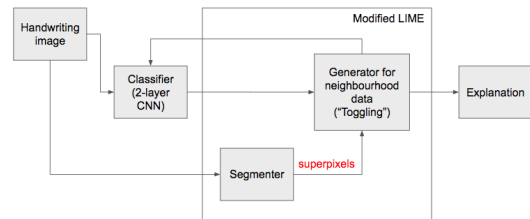


Figure 6: High level structure of our handwriting explainer

3.3.1 Modifying LIME for our application

LIME[16] generates explanations by toggling superpixels in the input image with some alternative (e.g. black pixels), and observing how the change affects the output of the classifier. LIME does this by generating a replacement image, for looking up alternative pixel values. Conventionally, LIME takes in a superpixel and generates a mean in all three channels (RGB), then writes this mean to the corresponding pixels in the replacement image.

This works great for natural images - the details in the eye here are removed so that the output of the classifier changes, which indicates that the concerned superpixel contributes to the classification. For our application, this is not the case - primarily as the image is binary and our features are not in color, but edges and some high dimensional features. The issue with using this for our explanations is that this creates a replacement that has an average value of the neighbouring pixels, and since the proportion of white background is so large, the replacement is mostly white. Thus, when the toggling occurs, it is toggling the white in the image with “mostly white” replacement image- which is not what we want in order to generate a good explanation.

LIME also takes an optional argument, `hide_color`, which specifies a flat colour for the entire replacement image. When used in our case, a good choice would be the background colour, black. Using a black replacement image allows the explainer to test if white segments should be turned to black,

while black segments are not changed by toggling. This corresponds to testing only the written areas, for whether they should be kept or erased.

To improve on the existing options offered by LIME, we made a small change in the LIME source code to handle two other options for `hide_color`, “reverse” and “reverse_nearest”. In the “reverse” option, we take the image to be tested and create a negative of that image. Thus, our replacement image is now black ink and white background, opposite to the original image. This ensures that all the pixels will be toggled to their opposite, to see the effect on the classifier. This allows us to treat the handwriting problem as a binary problem for each segment - “Should this part be written to, or not?” The explainer can then test for changes in the classification output when black segments are turned white, and when white segments are turned black.

Comparing the “reverse” option with the black replacement image, the “reverse” option offers more information, as it tests whether to keep or write to background segments, as well as whether to keep or erase written segments. However, unlike the black replacement image, the “reverse” option tests if background pixels far from the written sections should be written to as well, even if such edits are not of interest for improving handwriting.

To combine the benefits of the black replacement image and the “reverse” option, we created a “reverse_nearest” option that uses a Gaussian filter to blur the inverted image, and sets all pixels not affected by the blur to the background colour, black. This new replacement image does not store the opposites of pixels that are too far from the actual ink. All the pixels that are far from the writing are black, as in the original image, and thus toggling them causes no effect. In a way, this is imposing domain knowledge to guide the classifier to generate explanations only in the vicinity of the ink. Thus, we focus on the pixels that contain the letters and the pixels around it as this is the area we expect to contain explanations about the presence and absence of good and bad features. A σ of 0.8 is used for the blur, which was found empirically to yield the best results, although this can vary with stroke thickness. The segmenter then selects the superpixels to be toggled.

3.3.2 Segmentation

Choice and tuning of the segmenter is a crucial part in generating good explanations. The segmenter is responsible for generating the superpixels for the toggling. Choosing the right type of segmenter and tuning it to the application is vital for generating interpretable explanations. This can be thought of as a way of “expert guidance” for the system- as the human expert knows what features the algorithm should be looking for, the nature and size of the superpixels and roughly the number of superpixels in the image.

SLIC [1] was found to be ideal for segmentation of our MNIST dataset. SLIC uses a k-means style algorithm for generating the superpixels. The algorithm initializes K cluster centers, where K is a user input, and creates roughly same size superpixels around these cluster centers. The proximity measure m controls the compactness of the superpixels. Higher m makes the superpixels squarer as the proximity is weighted heavily.

We found that the best number of superpixels used and their compactness are very dependent on the size of the image, as well as the thickness of the strokes. For the low res-

olution MNIST image (28x28), the segmentation works well with high compactness of 50 (square superpixels are good as the image does not have much high dimensional features) and a moderate number of segments.

For the much higher resolution (40x73), we chose the Felzenszwalb segmenter instead (`min_size=2` `scale=0.01`, $\sigma = 0.8$), for the following reasons:

- There exist edges and especially curves that the segmenter with a square superpixel cannot segment correctly
- It is much faster, as the clustering process of the SLIC algorithm takes longer for a larger image.
- It produces thin curved segments that wrap the letter strokes, and toggling these can correspond to shifting the strokes slightly for better spacing or slant.

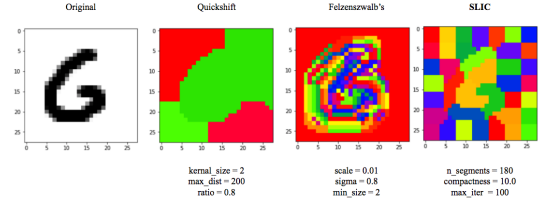


Figure 7: Comparison of the segmenters for the MNIST sample

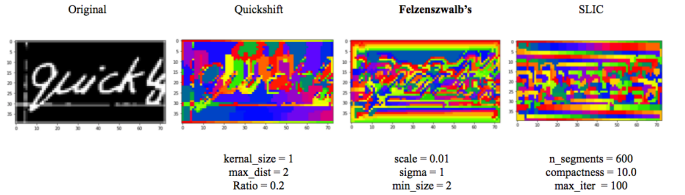


Figure 8: Comparison of the segmenters for the handwritten word “quickly” sample

3.3.3 Generating Explanations

For any sample image, we can generate an explanation by running the `LimeImageExplainer` method, `explain_instance`, with the desired segmenter and replacement image settings, as well as the number of perturbations to try. This returns an explanation object, which contains a list for each classification label, of segments sorted by their absolute contribution to the label, i.e. their influence on this outcome. The results can be displayed with the method `get_image_and_mask`.

How the explanation is displayed has a significant effect on the interpretability of the explanation results. We chose to visualise three aspects of the explanation:

1. Top contributing segments to the predicted label
2. Most influential segments for both labels
3. Result of suggested changes and segments to keep, based on most influential segments

In particular, the most influential segments were colour-coded as follows (see **Figure 9**):

- **Red (Good to change):** segments that contributed positively to the “bad” label, and segments that contributed negatively to the “good” label
- **Green (Good to keep):** segments that contributed positively to the “good” label, and segments that contributed negatively to the “bad” label

To visualise the top contributing segments, we can either opt to show the top `num_segments` segments by absolute contribution value (this can be filtered to show positive contributions only), or set a threshold that selects all segments with contributions above `min_weight`.

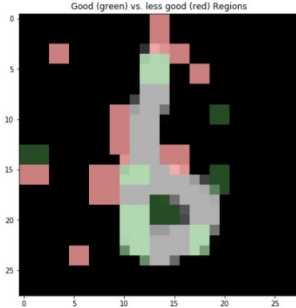


Figure 9: A six from MNIST which looks like the letter “b” and is classified “bad”. Red regions highlight areas to change, and green areas to keep.

4. RESULTS

We aim to offer explanation about *why* writing is classified as good or bad, and how to make bad writing better. Therefore we gave our classifier examples classified as good, to explain what features contribute to this and should be kept, and bad examples to explain what features are bad and what missing features should be added to improve the writing.

4.1 MNIST

We fed in several bad examples of the number six based upon our rules described in **Figure 3**.

4.1.1 How should I change my bad six?

Figure 10 shows different examples of “bad” sixes according to our rules laid out in **Figure 3**. All were correctly classified as “bad”. The left hand columns show reasons that contributed to this prediction; the central columns show recommended changes as well as what to keep the same; and the right hand column shows the resulting image after applying the recommended changes or retentions. At this point, the thresholds `num_segments` and `min_weight` had to be set manually to get the best interpretable result for each sample.

From seeing **Figure 10** we can infer the following suggestions from the explainer:

- Add an eye and extend the ascender in **Figure 11a**
- Change the curve joining the eye and adjust the ascender in **Figure 11b**
- Add curvature to the ascender in **Figure 10c**
- Close the eye in **Figure 10d**

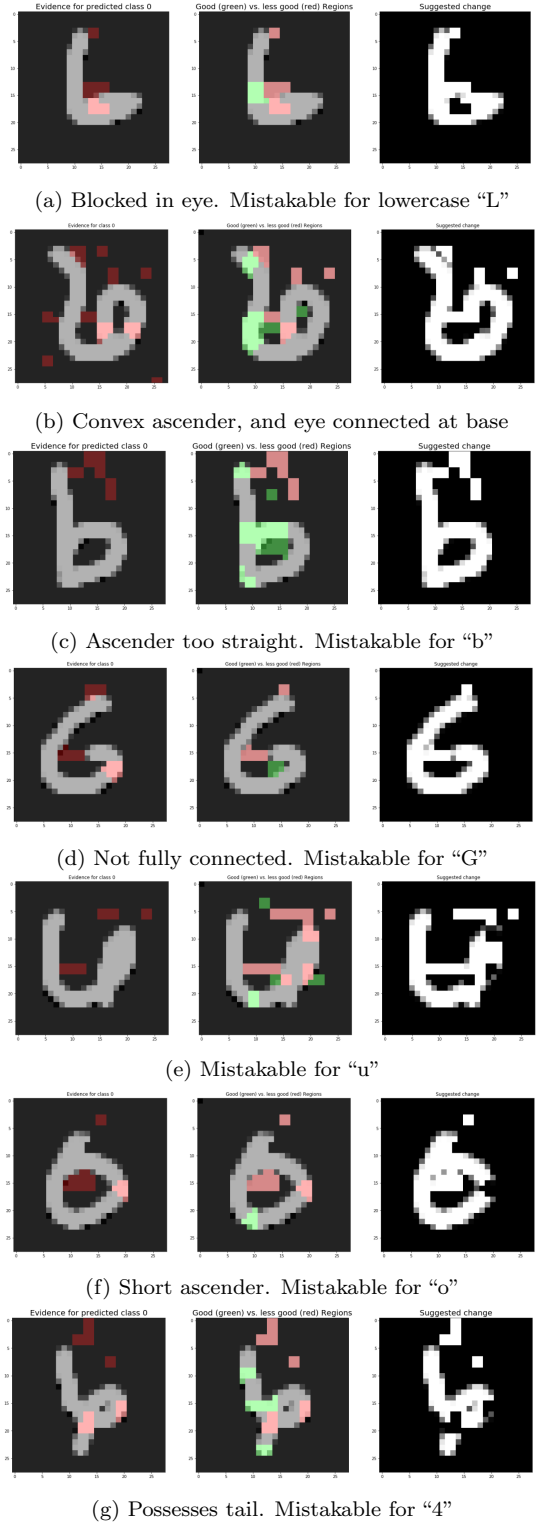


Figure 10: MNIST Sixes classified as “bad”

- Create an eye in **Figure 10e**. This case proves slightly unsuccessful potentially due to the number of changes necessary to produce a correct result
- Create an ascender in **Figure 10f**
- Create an ascender in **Figure 10g**. This case proves slightly unsuccessful potentially due to the number of changes necessary to produce a correct result

From these examples in **Figure 10**, it is possible to see that the explainer corrected the mistakes identified in our labelling stage.

4.1.2 Why is my six so great?

Additionally, it is interesting to know why your handwriting is classified as good, to assist in maintaining good habits.

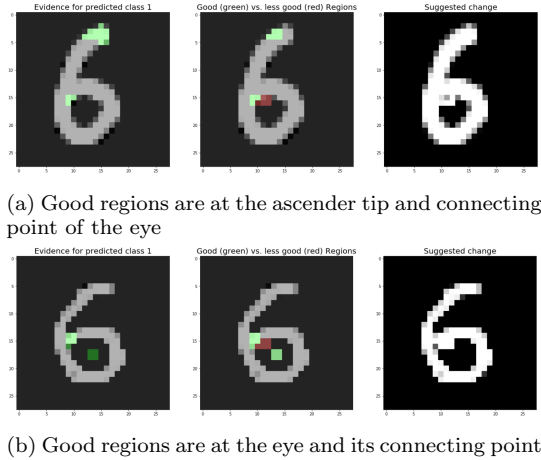


Figure 11: MNIST Sixes classified as “good”

Figure 11 shows an examples of sixes classified as “good”, where the explainer seems to infer that it is good to keep the curved tip of the ascender, keep the eye of the six empty, and connect the eye properly.

4.2 Handwritten Word Dataset

The handwritten word dataset has greater levels of complexity than MNIST, but a far smaller number of samples.

For the quickly dataset, the resulting explanations sometimes seem to pick out regions of letters that need correction. For example it would recommend deleting the tick of the letter “q” if it is too long (see **Figure 12**). Yet there is excess noise, meaning any valid explanations are sometimes difficult to discern (see **Figure 13**).

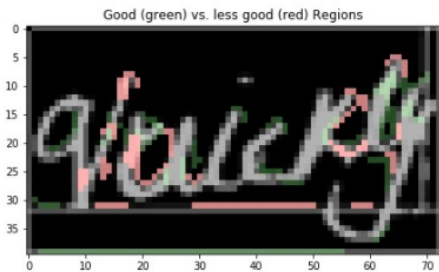


Figure 12: Explanation of a “bad” quickly

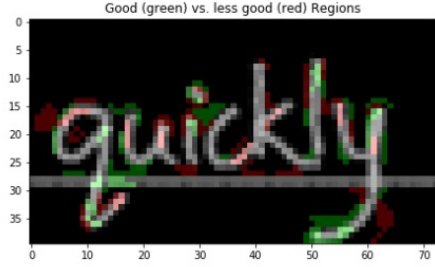


Figure 13: Explanation of a “good” quickly

The handwritten word “studio” data set generated within our project performed better in the explanation because it contains larger number of samples, is more balanced between classes, and was engineered to represent variation more evenly. **Figure 14** shows that the explainer suggested to straighten the letter “d” and add the dot for letter “i”. There’s also a light indication on closing the loop of letter “o”. These suggestions are relevant to the form of the letters. There are also suggestions to add lines below the floating “u” and “o”, which are relevant to alignment. The above results match the form and alignment requirements of the Minnesota Handwriting Assessment. The “improved” handwriting achieves 99.7178% towards the “good” class as comparing to 74.6325% for the original handwriting.



Figure 14: Explanation of a “good” quickly

5. LIMITATIONS AND FUTURE WORK

In this section we will put forth some of the limitations of our current system and the possible future directions.

The dataset was a major hindrance to our system since it was quite small thus could not represent all possible cases. Due to the shortcomings of the dataset both in quantity and quality, the classifier could not be tuned as accurately which directly affected the output of the explainer. Additionally, existing datasets have a lot of artifacts and noise. To overcome these limitations a considerably large and cu-

rated dataset could be created and tested upon. For labelling, we could use input from several people (or use Mechanical Turk) and follow majority consensus to reduce personal bias.

Currently we are using raw pixel values as features for the classifier and this may not be an ideal choice. It would be an interesting future direction to exploit feature engineering techniques to train the classifier. Aspects such as slant, spacing, stroke curvature, and relative size of characters, could help the classifier to better learn the rules mentioned in Section 3.1.

In the step for visualising explanations, the thresholds `num_segments` and `min_weight` had to be set manually to get the best interpretable result for each sample. A general pattern here was that as more segments were displayed, the changes suggested would increase the classifier’s score for the positive class, up to a point where any additional changes would add noise and worsen the prediction. In a scenario where the classifier is highly accurate and its score is reliable, a possible modification could be to vary `num_segments` and `min_weights`, produce suggested images, and pass the images into the classifier for a prediction. We can then plot the classifier score for the positive class against the respective thresholds, and select the threshold where the classifier score peaks. **Figure 14** shows a sample plot of classifier score against `num_segments`, for the example in **Figure 10e**.

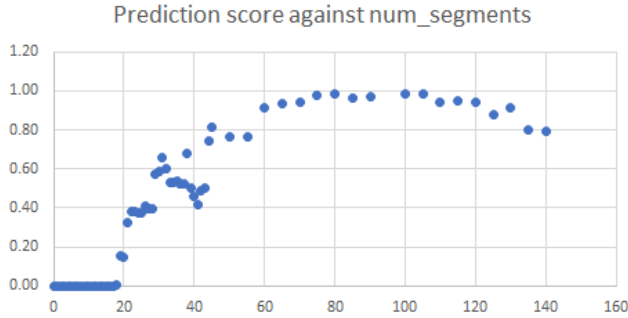


Figure 15: Plot of classifier score for positive class against `num_segments`

In addition, we can set a target classifier score, and count the number of changes needed to reach that score. This could be another interpretability feature, similar to an edit-distance metric that indicates how far the sample is from a good score.

Interlacing the current LIME structure with a segmenter which better follows the contours and curvature of the characters in a text, such as SCALP [9] could be a probable future work. Furthermore, exploring an alternative explainer to LIME such as “Deep Visualization Toolbox” [23], DeepLift [19], Integrated Gradients [21], or SHAP [13] could be helpful to improve the system performance.

To reveal the latent variables and provide explanations in human-understandable language, counter-factual explanations [22] such as “Your handwriting would be neater if your characters were more evenly spaced and sized” could be added to make the system user friendly.

6. CONCLUSION

We have created a system that aims to explain the rules learnt by the machine learning model which distinguishes between good and bad handwriting. One application area of this research could be the education sector where this system could help children to improve their handwriting thereby enabling educators to focus on other academic aspects of the individuals. Self-learning in the visual art forms of writing such as calligraphy and typography could become very convenient and interactive with the use of such systems. Explaining the working of the classifier while assessing handwriting would help users or administrators to establish faith in the system. Given enough varied data, it is possible for such a system to generate improvements for personalized styles of handwriting, unlike teachers in school who often drive students towards their “model” template for good handwriting. Thus, an explainable and transparent AI that can give user-specific suggestions for improvements can be created, which would enable seamless and easier learning transitions.

APPENDIX

A. GROUP DELEGATION OF WORK

:

:

:

:

B. REFERENCES

- [1] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Süsstrunk. Slic superpixels compared to state-of-the-art superpixel methods. *IEEE transactions on pattern analysis and machine intelligence*, 34(11):2274–2282, 2012.
- [2] A. Agius, M. Morelato, S. Moret, S. Chadwick, K. Jones, R. Eppe, J. Brown, and C. Roux. Dataset of coded handwriting features for use in statistical modelling. *Data in brief*, 16:1010–1024, 2018.
- [3] A. Agius, M. Morelato, S. Moret, S. Chadwick, K. Jones, R. Eppe, J. Brown, and C. Roux. Using handwriting to infer a writer’s country of origin for forensic intelligence purposes. *Forensic science international*, 282:144–156, 2018.
- [4] E. Aksan, F. Pece, and O. Hilliges. DeepWriting: Making Digital Ink Editable via Deep Generative Modeling. In *SIGCHI Conference on Human Factors in Computing Systems*, CHI ’18, New York, NY, USA, 2018. ACM.
- [5] S. Belongie, J. Malik, and J. Puzicha. Shape matching and object recognition using shape contexts. *IEEE transactions on pattern analysis and machine intelligence*, 24(4):509–522, 2002.
- [6] M. Bouillon and E. Anquetil. Handwriting analysis with online fuzzy models. In *17th Biennial Conference of the International Graphonomics Society (IGS)*, 2015.

- [7] T. H. Falk, C. Tam, H. Schellnus, and T. Chau. On the development of a computer-based handwriting assessment tool to objectively quantify handwriting proficiency in children. *Computer methods and programs in biomedicine*, 104(3):e102–e111, 2011.
- [8] C. T. Fuentes, S. H. Mostofsky, and A. J. Bastian. Children with autism show specific handwriting impairments. *Neurology*, 73(19):1532–1537, 2009.
- [9] R. Giraud, V.-T. Ta, and N. Papadakis. Scalp: Superpixels with contour adherence using linear path. In *Pattern Recognition (ICPR), 2016 23rd International Conference on*, pages 2374–2379. IEEE, 2016.
- [10] M.-L. Kaiser, J.-M. Albaret, and P.-A. Doudin. Relationship between visual-motor integration, eye-hand coordination, and quality of handwriting. *Journal of Occupational Therapy, Schools, & Early Intervention*, 2(2):87–95, 2009.
- [11] Kraggle. Handwritten Words Dataset. Retrieved from: <https://www.kaggle.com/nabeel965/handwritten-words-dataset/version/1>.
- [12] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [13] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, pages 4768–4777, 2017.
- [14] J. Reisman. Minnesota Handwriting Assessment. *San Antonio, TX: Psychological Corporation*, 1999.
- [15] J. E. Reisman. Development and reliability of the research version of the minnesota handwriting test. *Physical & Occupational Therapy in Pediatrics*, 13(2):41–55, 1993.
- [16] M. T. Ribeiro, S. Singh, and C. Guestrin. Why Should I Trust You?: Explaining the Predictions of any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144. ACM, 2016.
- [17] K. L. Roston, J. Hinojosa, and H. Kaplan. Using the minnesota handwriting assessment and handwriting checklist in screening first and second graders’ handwriting legibility. *Journal of Occupational Therapy, Schools, & Early Intervention*, 1(2):100–115, 2008.
- [18] scikit image. Comparison of segmentation and superpixel algorithms. http://scikit-image.org/docs/dev/auto_examples/segmentation/plot_segmentations.html, 2017. Last update 2017-09.
- [19] A. Shrikumar, P. Greenside, and A. Kundaje. Learning important features through propagating activation differences. *arXiv preprint arXiv:1704.02685*, 2017.
- [20] A. Stuhlsatz, J. Lippel, and T. Zielke. Feature extraction with deep neural networks by a generalized discriminant analysis. *IEEE transactions on neural networks and learning systems*, 23(4):596–608, 2012.
- [21] M. Sundararajan, A. Taly, and Q. Yan. Axiomatic attribution for deep networks. *arXiv preprint arXiv:1703.01365*, 2017.
- [22] S. Wachter, B. Mittelstadt, and C. Russell. Counterfactual explanations without opening the black box: Automated decisions and the gdpr. 2017.
- [23] J. Yosinski, J. Clune, A. Nguyen, T. Fuchs, and H. Lipson. Understanding neural networks through deep visualization. In *Deep Learning Workshop, International Conference on Machine Learning (ICML)*, 2015.
- [24] L. M. Zintgraf, T. S. Cohen, T. Adel, and M. Welling. Visualizing deep neural network decisions: Prediction difference analysis. *arXiv preprint arXiv:1702.04595*, 2017.
- [25] C. L. Zitnick. Handwriting beautification using token means. *ACM Trans. Graph.*, 32(4):53:1–53:8, July 2013.